Data Frames

DATA 118 Lab — Student Version

Goals

- To continue practicing with data frames, including their attributes, methods, and `pandas` functions.
- To carry out basic exploratory data analysis with aggregation functions and `.groupby()`.
- To examine relationships between variables using `pivot_table()`.
- To join data frames using `.merge()`.

Introduction to DataFrames

One of the most commonly used collections of data in data science is the DataFrame. Let's do a quick introduction before we get into working with them.

Different objects for different goals

There are different collections of data in Python because each works well in different contexts. We've already talked about lists and arrays:

Method 1: Lists (built-in to Python)

- Ordered and indexed
- Powerful built-in methods for sequences of data
- Flexible data types

Method 2: Arrays (must import `numpy`)

- All values of same data type
- Optimized for mathematical and matrix manipulations
- No row or column names

DataFrames two-dimensional tabular data structures that are especially useful if we have multiple observations of data of different types, or if we'd like for the data to be labeled.

Method 3: Data frames (must import `pandas`)

- Different columns can have different types
- Easy to manipulate by name
- Row and column names are built in

- Rows are "instances" as a `Series` object
- Columns are attributes of those individuals

Setup

For this lab we will be using the `pandas` library in Python, as well as a new dataset containing information on various Star Wars characters from Episodes I-VII. We have several data frames to use as a part of the larger dataset–run the cell below to setup the environment.

Exercise

```
import pandas as pd

# To download any of these data files go to
# https://posit.ds.uchicago.edu/correlationlab/www/<name of file
 ↪  here>.csv

starwars = pd.read_csv("./data/starwars.csv")
```

Data frame attributes

First, let's review some basic `pandas` using the `starwars` data frame. `starwars` includes the following variables:

- `name`: Name of the character
- `height`: Height (cm)
- `mass`: Weight (kg)
- `hair_color`, `skin_color`, `eye_color`: Hair, skin, and eye colors
- `birth_year`: Year born (BBY = Before Battle of Yavin)
- `sex`: The biological sex of the character, namely male, female, hermaphroditic, or none (as is the case for Droids)
- `gender`: The gender role or gender identity of the character as determined by their personality or the way they were programmed (as is the case for Droids)
- `homeworld`: Name of homeworld
- `species`: Name of species

Question 1. Which data frame attribute can confirm the names of these columns?

☐ `.columns`
☐ `.dtypes`
☐ `.index`
☐ `.shape`
☐ `.size`

Question 2. Which data frame attribute can help you find the number of observations in `starwars`?

☐ `.columns`
☐ `.dtypes`
☐ `.index`
☐ `.shape`
☐ `.size`

Use the method you chose above on our data frame.

Exercise — write your code here:

Question 3. How many observations are there in `starwars`?

☐ 11
☐ 87
☐ 1044

Discuss with a neighbor (or on Ed):

In the exercise above we asked you to use an attribute to find the number of observations in the data frame. It is also possible to use a function we have seen before. Can you guess what it is? Try to use it and discuss the difference in output with your neighbors.

Now let's find out what the `.dtypes` attribute does:

Exercise

```
starwars.dtypes
```

Remember that you can always look at the documentation for a function, method, or attribute–including the documentation for `.dtypes`. Hopefully after reading, you have found that this attribute is a `Series` object that describes the data found in each column.

Question 4. Which of the variables in `starwars` are numeric? Select all that apply.

☐ `name`
☐ `height`
☐ `mass`
☐ `hair_color`
☐ `skin_color`
☐ `eye_color`
☐ `birth_year`
☐ `sex`
☐ `gender`
☐ `homeworld`
☐ `species`

Categorical variables are usually saved as strings, but all we can see is `object`. Let's investigate further. Using square brackets, extract the `name` column.

Important note: Throughout this lab, please use double quotes instead of single quotes when dealing with strings: e.g., "this" not 'this'.

Exercise — write your code here:

Now, apply the `type()` function to the extracted column.

Exercise — write your code here:

Because it's a column of the data frame, `name` is saved as a `Series` object! But, we can still confirm that the `Series` is made up of strings. Write a line of code that checks the type of the first element in the column. Extract the column first, and then the row.

Exercise — write your code here:

You should finally see that this is indeed a string.

Now, write a line of code using double square brackets to select all of the numeric variables. Keep them in the same order as the original data frame.

Exercise — write your code here:

Write a line of code to extract the eleventh row of the data frame. Specify that we are selecting all columns.

Exercise — write your code here:

Remember that when indexing with square brackets in Python, the first index is inclusive and the second is exclusive. Write a line of code using `iloc` to select the tenth through nineteenth rows of the data frame. For this question, do not specify that we are selecting every column; `pandas` does this by default.

Exercise — write your code here:

Don’t forget, we can also subset the data frame using row extraction in conjunction with a logical operator to select the observations that meet a certain criteria.

Question 5. Which statement below tells us if each observation is a Droid?

☐ `starwars["species"] = Droid`
☐ `starwars["species"] == Droid`
☐ `starwars["species"] = "Droid"`
☐ `starwars["species"] == "Droid"`

Now, using your answer as well as `.loc`, extract the rows of `starwars` where the character is a Droid.

Exercise — write your code here:

Finally, write a line of code that will sort the data frame according to the character's mass. Write your code so that the largest masses appear first.

Exercise — write your code here:

Question 6. Which character has the largest mass?

☐ Captain Phasma
☐ Jabba Desilijic Tiure
☐ Yarael Poof
☐ Yoda

`.groupby()`

Now let's do some exploratory data analysis with `.groupby()` and our aggregation functions. Recall that the basic syntax is: `df.groupby("column_name")["value_column"].func()`. We like to use the following functions:

- `.count()`: The number of observations
- `.mean()`: The average of the observations
- `.min()`: The smallest observation
- `.max()`: The largest observation

Write a line of code to find out how many of each species there are in the data frame. Since everyone likely is labeled with a name, let's use `name` as the `value_column`.

Exercise — write your code here:

Discuss with a neighbor (or on Ed):
Note that while we are asking you to use `.groupby()` to complete these problems, we can also `.value_counts()`. See if you can make it work, and discuss the difference in output with your neighbors.

Save your result as a new data frame, `starwars_species`. Right below it, write one new line of code to give us the number of species with more than one character in the *Star Wars* universe.

Exercise — write your code here:

Question 7. How many species are there with more than one character?
Answer:
_____

Using the original `starwars` data frame, write a single line of code to calculate the mean mass for each of the species. Normally we would not do this for very very small groups (e.g., one observation), but for now, it's okay.

Exercise — write your code here:

Save your result as a new data frame, `starwars_species_mass`. Then, write one line of code to sort the species that have an average mass of less than 100 kg from most to least massive.

Exercise — write your code here:

Pivot tables

Let's transition to working with pivot tables. Recall that when working with pivot tables we provide `DataFrame`, `index`, `columns`, `values`, and `aggfunc` arguments.

Write a line of code to count the number of characters with each eye and hair color combination. Eye color should be in the rows of your pivot table. Use "name" in the `values` argument.

Exercise — write your code here:

Question 8. How many characters have black eyes and no hair?
☐ 1.0
☐ 3.0
☐ 9.0
☐ `NaN`

This is what is known as a "sparse" table–lots of categories with no values, and only a few with actual numbers. Let's assign our pivot table to a data frame called `pivot_df`, and use a method called `.stack()` on it.

Exercise — write your code here:

This helped us a lot! It dropped all NaN values, and put it back in a tabular form.

Also take note of the warning given to us by Python. There isn't an issue with running the code now, but there will be in the future.

Discuss with a neighbor (or on Ed):
In the exercise above, we asked you to use `.stack()` on a pivot table to make it more readable. Could we have used `.groupby()` on the original `starwars` data frame instead? Would there be any difference in output?

Now let's take a look at a smaller, less sparse table.

Write a line of code to create a pivot table that compares mean heights grouped by sex and species. `species` should be in the rows of your table.

Exercise — write your code here:

Question 9. What is the average height of a female human (in cm)?
☐ `NaN`

`.merge()`

Finally, let's consider `.merge()`. We have two variables in the `pilots` data frame– a string containing all vehicles a character has driven, and a string containing all starships a character has driven.

Exercise

```
pilots = pd.read_csv("./data/pilots.csv", index_col = 0)
```

Write one line to confirm which columns `pilots` has in common with `starwars`. To do this, make both column `Series` into sets, and then take the intersection. Return to last week's tutorial if you forgot about sets!

Exercise — write your code here:

Question 10. Which column do `pilots` and `starwars` have in common?

☐ `height`
☐ `name`
☐ `vehicles`
☐ `starships`

Merge `pilots` and `starwars` into one data frame called `starwars_big` using `pd.merge()`. No need to add any additional arguments.

Exercise — write your code here:

If we take a look, there are quite a few cells in the data frame with missing values (`NaN`).

If a character has not driven any vehicles, they will have an empty string `""` in the `vehicles` column in the `pilots` data frame (same with `starships`). Write a line of code to take a subset of `pilots` where the characters are only included if they have driven at least one vehicle or at least one starship Save the subset as `pilots_small`.

Hint: Use the `pandas` function `notna()` in conjunction with a binary operator.

Exercise — write your code here:

Now, merge `pilots_small` and `starwars` in such a way that ONLY the characters who have driven a vehicle or starship remain in the dataset. Call this dataframe `starwars_pilots`.

Exercise — write your code here:

Thus far, we have only shown you how to use `merge()` as a function. However, we can also use it as a method, where it acts on one of the data frames explicitly. Take a look at this example:

Exercise

```
starwars_pilots_method = pilots_small.merge(starwars, how = "left")
```

Both data frames: `starwars_pilots` and `starwars_pilots_method` should be identical. Write one line of code to check this using `.equals()`.

Exercise — write your code here:

Now that we have learned how to join two data frames that have a column in common, let's take a look at how to add multiple data frames together when they all have the same columns. For example, we have seven smaller data frames containing all of the characters that appear in Episodes I-VII.

Exercise

```
ep1 = pd.read_csv("./data/the_phantom_menace.csv", index_col = 0)
ep2 = pd.read_csv("./data/attack_of_the_clones.csv", index_col = 0)
ep3 = pd.read_csv("./data/revenge_of_the_sith.csv", index_col = 0)
ep4 = pd.read_csv("./data/a_new_hope.csv", index_col = 0)
ep5 = pd.read_csv("./data/the_empire_strikes_back.csv", index_col = 0)
ep6 = pd.read_csv("./data/return_of_the_jedi.csv", index_col = 0)
ep7 = pd.read_csv("./data/the_force_awakens.csv", index_col = 0)
```

Similarly to strings and lists, we can concatenate data frames together. The function to do so is `concat()`. `concat()` accepts a list of data frames to be concatenated.

Concatenate the `ep1` and `ep2` data frames.

Exercise — write your code here:

You can also provide a longer list to concatenate more data frames simultaneously. Concatenate all seven of the smaller `epx` data frames.

Exercise — write your code here: